

ISG 算法插件开发规范

Version 2.0

覆盖普通插件、训练插件、数据集兼容性、模型权重、离线日志、标签转换、取消处理和
checkpoint 输出。

生成日期：2026-07-20

目录

1. 适用范围
 2. 注册和文件复制规则
 3. 最小文件结构
 4. PARAMETERS 字段规范
 5. payload 结构
 6. context 接口
 7. 普通插件返回规范
 8. 训练数据集要求
 9. 检测训练插件要求
 10. 模型权重管理
 11. 在线实验平台和离线运行
 12. 训练取消和进度
 13. 训练插件成功返回
 14. 错误返回
 15. 模板说明
 16. 发布前检查清单
- 附录 A：普通插件模板完整源码
- 附录 B：训练插件模板完整源码

ISG 算法插件开发规范 v2.0

1. 适用范围

ISG 算法插件是标准 Python 模块（.py 文件），通过软件“算法参数配置”页面注册。所有插件都必须声明模块级 `PARAMETERS`，并实现 `run(payload, context)`。

插件分为两类：

- 普通插件：生成、增强、清洗和评估等非训练任务，参考 `plugins/user/_TEMPLATE.py`。
- 训练插件：调用机器学习或深度学习框架训练模型，参考 `plugins/user/_TRAINING_TEMPLATE.py`。

训练插件除通用接口外，必须声明数据集要求，管理模型权重和训练数据转换，支持取消，并返回真实存在的 `checkpoint`。示例文字和占位路径不得作为运行实现。

2. 注册和文件复制规则

软件注册脚本插件时，只把选择的 .py 文件复制到 `plugins/user/`，不会自动复制以下内容：

- `.pt`、`.pth`、`.onnx`、`.engine` 等模型文件；
- 自定义 Python 包和相邻辅助脚本；
- 数据配置 YAML、词表、类别文件；
- DLL、CUDA 扩展或外部命令行程序。

因此插件依赖的资产必须采用以下一种方式：

1. 使用软件已经安装的 Python 包；
2. 将模型和配置放入软件约定的稳定资产目录，并使用相对软件根目录的路径解析；
3. 允许用户填写绝对路径，并在运行前检查文件；
4. 明确支持联网下载，同时给出离线环境下的错误信息。

换电脑时必须同时迁移插件、模型文件、配置文件和依赖。数据库中的算法记录不能代替这些实体文件。

3. 最小文件结构

```
# -*- coding: utf-8 -*-
from __future__ import annotations

from pathlib import Path
from typing import Any

PARAMETERS: list[dict[str, Any]] = [
    {
        "name": "threshold",
        "type": "float",
        "label": "阈值",
        "default": 0.5,
        "min": 0.0,
        "max": 1.0,
        "options": [],
        "description": "判定阈值",
        "required": False,
    },
]

def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    parameters = payload.get("parameters", {}) or {}
    output_dir = Path(payload["output"] ["output_dir"])
    output_dir.mkdir(parents=True, exist_ok=True)
    ...
```

无参数插件也必须声明：

```
PARAMETERS = []
```

4. PARAMETERS 字段规范

字段	类型	必填	说明
name	str	是	参数变量名，建议英文小写加下划线
type	str	是	string、int、float、bool 或 select
label	str	是	UI 显示名称
default	任意	是	默认值，必须与 type 匹配
min	number	否	int/float 最小值
max	number	否	int/float 最大值
options	list	否	select 候选项；非空时训练参数界面显示下拉框
description	str	否	参数用途和限制
required	bool	否	是否必填，默认 False

类型示例：

type	default 示例	说明
string	"normal"	普通文本、路径或设备字符串
int	100	整数
float	0.5	浮点数
bool	True	开关
select	"yolov8n.pt"	必须提供非空 options

不要使用 `number`、`integer`、`json` 等旧类型名。

5. payload 结构

```
payload = {
  "task_id": 42,
  "algorithm_key": "training.image.custom_detector",
  "category": "training",
  "modality": "image",
  "parameters": {
    "epochs": 100,
    "device": "0",
  },
  "input": {
    "dataset_id": 1,
    "dataset_path": "C:/data/datasets/demo",
    "samples": [
      {
        "id": 10,
        "name": "image_001.jpg",
        "path": "C:/data/datasets/demo/image_001.jpg",
        "sample_path": "C:/data/datasets/demo/image_001.jpg",
        "relative_path": "images/image_001.jpg",
        "status": "active",
        "metadata": {"annotation_format": "COCO"},
        "labels": [
          {
            "type": "detection",
            "class_name": "ship",
            "bbox": [0.5, 0.5, 0.3, 0.2],
            "bbox_format": "xcywh_normalized",
          }
        ]
      }
    ],
  },
  "output": {
    "output_dir": "C:/data/tasks/42/output",
  },
  "target_count": 100,
}
```

插件应优先读取 `sample["path"]` 或 `sample["sample_path"]`。数据库标签通过 `sample["labels"]` 传入，不应再直接访问数据库或假设每张图片只有一个标签。

6. context 接口

方法	说明
<code>context.set_progress(percent, message)</code>	上报 0 到 100 的进度和状态文本
<code>context.log(level, message, payload)</code>	写入结构化日志，level 为 info/warn/error
<code>context.is_cancel_requested()</code>	检查主动停止请求

耗时循环、每个 epoch 或每批次都应检查取消：

```
if context.is_cancel_requested():
    return {
        "ok": False,
        "error_code": "CANCELLED",
        "message": "任务已取消",
    }
```

插件收到取消请求后应尽快停止，不得继续把不完整模型标记为成功。

7. 普通插件返回规范

生成或增强插件：

```
return {
  "ok": True,
  "outputs": [
    {
      "source_sample_id": sample.get("id"),
      "output_path": str(output_path),
      "relative_path": output_path.name,
      "metadata": {"method": "custom_augment"},
      "status": "created",
      "labels": transformed_labels,
      "label_policy": "transformed",
    }
  ],
  "logs": [],
}
```

检测数据发生裁剪、缩放、翻转、旋转、透视或像素映射时，必须同步变换检测框并返回

`labels`。如果图像几何位置没有变化，可以省略 `labels` 或使用

`label_policy="inherit"`。不能变换标签的算法不得对检测数据做几何变化。

清洗建议插件：

```
return {
  "ok": True,
  "suggestions": [
    {
      "sample_id": 1,
      "issue_type": "blur",
      "suggested_action": "repair",
      "confidence": 0.85,
      "message": "图像存在模糊",
      "details": {},
    }
  ],
  "logs": [],
}
```

8. 训练数据集要求

训练插件应声明模块级 DATASET_REQUIREMENTS:

```
DATASET_REQUIREMENTS = {
    "modalities": ["image"],
    "label_types": ["detection"],
    "min_samples": 10,
    "min_classes": 2,
    "min_samples_per_class": 2,
    "required_extensions": [".jpg", ".png"],
    "required_columns": [],
    "required_companion_roles": [],
    "min_complete_groups": 0,
    "allow_unlabeled": False,
    "bbox_required": True,
}
```

字段	说明
modalities	image、text、audio、tabular、multimodal、other
label_types	classification、detection、segmentation、none
min_samples	最低有效样本数
min_classes	最低类别数
min_samples_per_class	每个类别最低样本数
required_extensions	必须包含的扩展名
required_columns	表格/CSV 必须包含的列
required_companion_roles	多模态样本必须关联的角色
min_complete_groups	最低完整多模式组数
allow_unlabeled	是否允许无标签样本
bbox_required	是否必须包含有效检测框

复杂条件可以增加:

```
def validate_dataset(summary: dict[str, Any]) -> dict[str, Any]:
    compatible = summary.get("sample_count", 0) >= 10
    return {
        "compatible": compatible,
        "reason": "至少需要 10 个有效样本",
    }
```

软件会先按 DATASET_REQUIREMENTS 过滤数据集，再在创建任务和执行训练前复检。

9. 检测训练插件要求

YOLOv、Detectron2 等目标检测训练插件必须遵守:

1. 全量扫描所有样本标签，建立一次全局类别映射，例如 {"猫": 0, "狗": 1}。
2. 所有图片共用同一份类别映射，禁止按图片重新从 0 编号。
3. 一张图片可以包含多个类别和多个框。
4. 划分训练、验证和测试集时，应同时考虑每个类别的标注数量。
5. 框坐标必须校验并转换为目标框架需要的格式。
6. 转换后的图片、标签和 YAML 必须写入当前任务的 output_dir。

7. 不得修改原始数据集文件。

YOLO 格式示例：

```
0 0.500000 0.500000 0.300000 0.200000
1 0.250000 0.400000 0.100000 0.150000
```

其中每一行依次为：class_id center_x center_y width height，坐标范围必须在 0 到 1。

10. 模型权重管理

训练插件必须明确模型来源：

- 本地预训练：运行前检查权重是否存在；
- 从头训练：使用模型结构 YAML 或代码配置；
- 自动下载：在参数说明和日志中明确提示，需要处理无网络情况；
- 用户路径：支持绝对路径并验证扩展名和文件存在性。

例如 Ultralytics：

```
model_name = str(parameters.get("model", "yolov8n.pt"))
model_path = resolve_model_path(model_name)
model = YOLO(model_path)
```

`YOLO("yolov8n.pt")`

在本地找不到权重时会联网下载。下载的是预训练起点，不是当前数据集训练完成后的模型。训练结果通常位于 `runs/train/weights/best.pt`，必须返回该真实路径。

插件不得把 YOLOv8 权重交给 YOLOv5 引擎，也不得假设不同框架的 `.pt` 文件可以互换。

11. 在线实验平台和离线运行

桌面本地训练默认不依赖 Comet、WandB、ClearML、MLflow、Neptune 等在线平台。Ultralytics

插件建议在创建模型前关闭未使用的集成：

```
from ultralytics import YOLO, settings

settings.update({
    "comet": False,
    "wandb": False,
    "clearml": False,
    "mlflow": False,
    "neptune": False,
    "tensorboard": False,
})

model = YOLO("yolov8n.pt")
```

只有用户明确启用并提供 API Key 时，插件才应连接在线平台。缺少 API Key 不应导致本地训练失败。

12. 训练取消和进度

框架提供 callback 时，应把取消和进度接入 callback：

```
def on_train_epoch_end(trainer):
    if context.is_cancel_requested():
        trainer.stop = True
    current = int(trainer.epoch) + 1
    context.set_progress(current / epochs * 100, f"Epoch {current}/{epochs}")

model.add_callback("on_train_epoch_end", on_train_epoch_end)
```

13. 训练插件成功返回

训练成功必须返回真实存在的模型文件：

```
checkpoint = Path(real_best_checkpoint)
if not checkpoint.is_file():
    return {
        "ok": False,
        "error_code": "CHECKPOINT_NOT_FOUND",
        "message": f"checkpoint 不存在: {checkpoint}",
    }

return {
    "ok": True,
    "outputs": [
        {
            "artifact_path": str(checkpoint),
            "metadata": {
                "backbone": "yolov8",
                "class_names": class_names,
                "class_to_id": class_to_id,
                "epochs": epochs,
                "device": device,
                "data_yaml": str(data_yaml),
            },
        },
    ],
    "logs": [f"training complete: {checkpoint}"],
}
```

禁止返回：

- "训练得到的best.pt" 等示意文字；
- 未创建的相对路径；
- 临时目录中已经删除的文件；
- 与当前任务无关的旧 checkpoint。

14. 错误返回

```
return {
    "ok": False,
    "error_code": "NO_INPUT_SAMPLES",
    "message": "无输入样本",
}
```

推荐错误码：

error_code	场景
NO_INPUT_SAMPLES	没有输入样本
INVALID_LABEL	标签格式或坐标无效
MODEL_NOT_FOUND	模型文件不存在
DEPENDENCY_MISSING	Python 包或外部程序缺失
TRAINING_FAILED	框架训练失败
CHECKPOINT_NOT_FOUND	训练完成但模型文件不存在
CANCELLED	用户主动停止
PLUGIN_NOT_IMPLEMENTED	模板中的训练逻辑尚未实现

QML 接口只显示简洁 `message`，插件应在 `message` 或服务端日志中保留足够的诊断信息。

15. 模板说明

普通插件模板

文件: `plugins/user/_TEMPLATE.py`

适用于生成、增强、清洗和普通评估。模板演示参数读取、样本遍历、输出文件、进度和取消处理。

训练插件模板

文件: `plugins/user/_TRAINING_TEMPLATE.py`

模板可以被反射，但默认返回 `PLUGIN_NOT_IMPLEMENTED`。开发者必须实现 `_train_model()`，确保返回真实 `checkpoint` 后才能注册为可用训练算法。

对于 YOLOv8，可参考 `plugins/user/yolov8.py` 的完整数据转换、离线日志、设备选择、取消和 `checkpoint` 返回实现。

16. 发布前检查清单

- [] `PARAMETERS` 位于模块顶层，字段类型正确。
- [] `run(payload, context)` 可以被导入。
- [] 训练插件声明了准确的 `DATASET_REQUIREMENTS`。
- [] 插件没有占位数据路径和占位 `checkpoint`。
- [] 模型文件缺失时有明确处理策略。
- [] 本地模式不会因为 Comet 等平台缺少 API Key 而失败。
- [] 标签读取自 `samples[*].labels`，支持多类别和多框。
- [] 训练数据转换写入任务 `output_dir`，不修改原始数据。
- [] 长任务能上报进度并响应取消。
- [] 成功返回的 `artifact_path` 在返回前确实存在。
- [] 换电脑部署时包含插件、模型资产、配置和依赖。
- [] 已使用小数据集完成至少一次端到端训练测试。

附录 A：普通插件模板完整源码

文件：plugins/user/_TEMPLATE.py

```
# -*- coding: utf-8 -*-
"""普通算法插件模板（生成、清洗、评估等非训练插件）。

复制后改文件名、PARAMETERS 和 run() 即可在算法配置页注册。
训练算法请使用同目录的 _TRAINING_TEMPLATE.py。
"""

from __future__ import annotations

from pathlib import Path
from typing import Any

PARAMETERS: list[dict[str, Any]] = [
    {
        "name": "threshold",
        "type": "float",
        "label": "阈值",
        "default": 0.5,
        "min": 0.0,
        "max": 1.0,
        "options": [],
        "description": "判定阈值, 范围 0~1",
        "required": False,
    },
    {
        "name": "enable_feature",
        "type": "bool",
        "label": "启用增强",
        "default": True,
        "options": [],
        "description": "是否启用额外处理",
        "required": False,
    },
    {
        "name": "mode",
        "type": "select",
        "label": "处理模式",
        "default": "normal",
        "options": ["normal", "aggressive", "conservative"],
        "description": "算法处理强度模式",
        "required": False,
    },
]

def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    """算法执行入口。"""
    parameters = payload.get("parameters", {}) or {}
    output_dir = Path(payload.get("output", {})).get("output_dir") or "."
    output_dir.mkdir(parents=True, exist_ok=True)

    samples = payload.get("input", {}).get("samples", []) or []
    if not samples:
        return {"ok": False, "error_code": "NO_INPUT_SAMPLES", "message": "无输入样本"}

    target_count = max(1, int(payload.get("target_count") or len(samples)))

    threshold = float(parameters.get("threshold", 0.5))
    enable_feature = _as_bool(parameters.get("enable_feature", True))
    mode = str(parameters.get("mode", "normal"))

    outputs: list[dict[str, Any]] = []
    for index in range(target_count):
        if context.is_cancel_requested():
```

```
        return {"ok": False, "error_code": "CANCELLED", "message": "任务已取消"}

    sample = samples[index % len(samples)]
    source_path = Path(sample.get("sample_path") or sample.get("path") or "")
    if not source_path.exists():
        continue

    import shutil
    output_path = output_dir / f"{source_path.stem}_out_{index:04d}{source_path.suffix
        or '.dat'}"
    shutil.copy2(source_path, output_path)

    outputs.append({
        "source_sample_id": sample.get("id"),
        "output_path": str(output_path),
        "relative_path": output_path.name,
        "metadata": {
            "method": "custom_algorithm",
            "algorithm_key": payload.get("algorithm_key", ""),
            "parameters": {"threshold": threshold, "enable_feature": enable_feature,
                "mode": mode},
        },
        "status": "created",
    })
    context.set_progress((index + 1) * 100 / target_count, f"已处理 {index +
        1}/{target_count}")

    return {"ok": True, "outputs": outputs, "logs": []}

def _as_bool(value: Any) -> bool:
    """把前端传来的开关值转成 bool。"""
    if isinstance(value, bool):
        return value
    return str(value).strip().lower() in {"1", "true", "yes", "y", "是"}
```

附录 B：训练插件模板完整源码

文件：plugins/user/_TRAINING_TEMPLATE.py

```
# -*- coding: utf-8 -*-
"""训练算法插件模板。

本文件可以被软件反射和注册，但必须实现 _train_model() 后才能真正训练。
不要把示例路径或占位 checkpoint 当成真实训练产物返回。
"""

from __future__ import annotations

from pathlib import Path
from typing import Any

PARAMETERS: list[dict[str, Any]] = [
    {
        "name": "model_path",
        "type": "string",
        "label": "模型或预训练权重",
        "default": "",
        "options": [],
        "description": "使用稳定目录或绝对路径；插件注册只复制 .py，不复制模型文件",
        "required": False,
    },
    {
        "name": "epochs",
        "type": "int",
        "label": "训练轮次",
        "default": 100,
        "min": 1,
        "max": 1000,
        "options": [],
        "description": "训练 epoch 数量",
        "required": False,
    },
    {
        "name": "device",
        "type": "string",
        "label": "训练设备",
        "default": "",
        "options": ["", "0", "1", "0,1", "cpu"],
        "description": "支持自动、单显卡、双显卡和 CPU",
        "required": False,
    },
]

# 必须按算法真实输入填写。软件注册时会自动读取并用于数据集筛选和训练前复检。
DATASET_REQUIREMENTS = {
    "modalities": ["image"],
    "label_types": ["detection"],
    "min_samples": 2,
    "min_classes": 1,
    "min_samples_per_class": 1,
    "required_extensions": [".jpg", ".jpeg", ".png"],
    "required_columns": [],
    "required_companion_roles": [],
    "min_complete_groups": 0,
    "allow_unlabeled": False,
    "bbox_required": True,
}

def validate_dataset(summary: dict[str, Any]) -> dict[str, Any]:
    """可选复杂校验；简单条件优先写在 DATASET_REQUIREMENTS。"""
```

```

compatible = int(summary.get("sample_count", 0)) >= 2
return {"compatible": compatible, "reason": "训练至少需要 2 个有效样本"}

def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    """训练入口：准备数据、调用模型、返回真实 checkpoint。"""
    parameters = payload.get("parameters", {}) or {}
    samples = payload.get("input", {}).get("samples", []) or []
    output_dir = Path(payload.get("output", {}).get("output_dir") or ".")
    output_dir.mkdir(parents=True, exist_ok=True)

    if not samples:
        return {"ok": False, "error_code": "NO_INPUT_SAMPLES", "message": "无输入样本"}
    if context.is_cancel_requested():
        return {"ok": False, "error_code": "CANCELLED", "message": "任务已取消"}

    try:
        checkpoint_path, metadata = _train_model(
            samples=samples,
            parameters=parameters,
            output_dir=output_dir,
            context=context,
        )
    except NotImplementedError as exc:
        return {"ok": False, "error_code": "PLUGIN_NOT_IMPLEMENTED", "message": str(exc)}
    except Exception as exc:
        return {"ok": False, "error_code": "TRAINING_FAILED", "message": str(exc)}

    checkpoint = Path(checkpoint_path)
    if not checkpoint.is_file():
        return {
            "ok": False,
            "error_code": "CHECKPOINT_NOT_FOUND",
            "message": f"训练结束但 checkpoint 不存在: {checkpoint}",
        }

    context.set_progress(100.0, "训练完成")
    return {
        "ok": True,
        "outputs": [{
            "artifact_path": str(checkpoint),
            "metadata": dict(metadata or {}),
        }],
        "logs": [],
    }

def _train_model(
    *,
    samples: list[dict[str, Any]],
    parameters: dict[str, Any],
    output_dir: Path,
    context: Any,
) -> tuple[Path, dict[str, Any]]:
    """必须替换为真实训练实现。

    实现要求：
    1. 从 samples[*].path/sample_path 和 samples[*].labels 读取数据及标签。
    2. 如模型需要特定格式，在 output_dir 内生成转换后的训练数据。
    3. 周期性调用 context.is_cancel_requested() 和 context.set_progress()。
    4. 默认关闭 Comet、WandB、ClearML 等在线平台，除非用户显式启用。
    5. 返回真实存在的 checkpoint 路径和可序列化 metadata。
    """
    raise NotImplementedError("请先实现 _train_model(), 再注册此训练插件")

```